WEEK 14

Grover in Practice, and Shor's Algorithm

17, 19 November · Objectives 7, 8

Grover on hardware, then the algorithm that made quantum computing a policy question. Shor's is the one result in this course with a deadline attached to it.

Grover on hardware

The multi-controlled Z gates dominate cost, and their expense is easy to underestimate:

```
from qiskit import transpile

qc, k = grover(4, marked=11)
qc_hw = transpile(qc, backend, optimization_level=3)
print(f"logical depth {qc.depth()} -> physical {qc_hw.depth()}")
print(f"CX count: {qc_hw.count_ops().get('cx', 0)}")
# logical depth 61 -> physical 388
# CX count: 214
```

An n-controlled X decomposes into O(n) Toffolis, each needing 6 CNOTs, and every CNOT carries ~10^{-2} error. With 214 CX gates:

```
(1 - 0.01)^214 ≈ 0.115
```

About 12% of runs survive without a two-qubit error. Measured success on a real device runs 20–40% for n = 4 — better than the naive estimate, since not every gate error flips the outcome, but far from the simulator's 96.7%.

Ancilla-assisted decomposition trades qubits for depth:

```
qc.mcx(controls, target, mode='v-chain', ancilla_qubits=anc)
# O(n) CX and O(n) depth, using n-2 ancillas — versus O(n²) CX without them.
```

On NISQ hardware this trade is usually correct: idle ancillas decohere, but a shallower circuit decoheres less overall.

> **Analogy boundary — gate count as instruction count.** Counting gates like instructions gives a reasonable first cost model, and depth-versus-count maps onto latency-versus-throughput. It fails on **error compounding**: instructions do not have a probability of silently corrupting the machine. Success falls off *exponentially* in gate count, so a circuit 2× longer is not 2× slower — it may be qualitatively infeasible. There is no NISQ equivalent of "let it run longer."

Shor's algorithm

Problem. Factor a composite N. Best classical algorithm (general number field sieve) runs in sub-exponential time; Shor runs in **polynomial** time — $O((\log N)^3)$. This is an exponential speedup on a problem the world's public-key infrastructure depends on.

The reduction to period-finding. Factoring is not attacked directly. The structure:

```
 1. Pick random a with 1 < a < N.
2. If gcd(a, N) > 1, that gcd is a factor. Done (rare).
3. Find the PERIOD r of f(x) = aˣ mod N  —  the smallest r with aʳ ≡ 1 (mod N).
4. If r is odd, or a^{r/2} ≡ -1 (mod N), restart with a different a.
5. Otherwise gcd(a^{r/2} ± 1, N) are nontrivial factors.
```

Steps 1, 2, 4, and 5 are classical and cheap. **Only step 3 is quantum.** The algebra of step 5: if r is the period, then $a^r - 1 \equiv 0 \pmod N$, so N divides $(a^{r/2} - 1)(a^{r/2} + 1)$. Provided neither factor is a multiple of N, each shares a nontrivial divisor with N.

Worked example, N = 15, a = 7:

```
 7¹ = 7,  7² = 4,  7³ = 13,  7⁴ = 1  (mod 15)      →  r = 4
r even ✓,  7² = 4 ≢ -1 (mod 15) ✓
gcd(7² - 1, 15) = gcd(48, 15) = 3
gcd(7² + 1, 15) = gcd(50, 15) = 5          15 = 3 × 5  ✓
```

Why period-finding is quantum-hard

Classically, finding the period of a^x mod N requires evaluating f at exponentially many points — the period can be as large as N, and there is no structure a classical algorithm can exploit to locate it faster.

Quantumly, prepare a superposition over all x, compute f(x) into a second register, and the first register is left in a **periodic superposition**. The quantum Fourier transform then converts period into frequency, which measurement can read.

```
 1. |0⟩|0⟩  --H⊗ⁿ-->  (1/√Q) Σₓ |x⟩|0⟩
2. modular exponentiation:  (1/√Q) Σₓ |x⟩|aˣ mod N⟩
3. measuring the second register collapses the first to
   a comb of states spaced r apart:  |x₀⟩ + |x₀+r⟩ + |x₀+2r⟩ + ...
4. QFT⁻¹ maps the comb to peaks at multiples of Q/r
5. continued fractions on the measured value recovers r
```

Step 3 is worth pausing on: measuring the *second* register — which you never look at again — is what imposes periodicity on the first. This is a constructive use of collapse, and it is where partial measurement from Week 10 earns its keep.

> **Analogy boundary — the QFT as an FFT.** The QFT applies the same unitary as the DFT and does it in $O((\log Q)^2)$ gates versus the FFT's $O(Q \log Q)$ — apparently an exponential speedup at signal processing. It is not, and the reason matters: the QFT's output amplitudes are **not readable**. You get one sample from the transformed distribution, not the spectrum. The QFT is useful only when the answer is a *single dominant frequency* you can identify from few samples. Shor's works because period-finding has exactly that structure. You cannot use the QFT to accelerate signal processing.

Cryptographic consequences

PRIMITIVE	BASIS	QUANTUM STATUS
RSA	factoring	BROKEN (Shor, polynomial)
Diffie-Hellman	discrete log	BROKEN (Shor variant)
ECDSA / ECDH	elliptic-curve DL	BROKEN (Shor variant)
AES-128	brute force	weakened to ~2^{64} (Grover)
AES-256	brute force	~2^{128} — safe
SHA-256	preimage	~2^{128} — safe

The asymmetry is the point: **asymmetric cryptography falls exponentially, symmetric cryptography only quadratically.** Doubling symmetric key lengths restores their margin. There is no equivalent patch for RSA or ECC — they require replacement, which is what NIST's post-quantum standards (ML-KEM, ML-DSA, SLH-DSA, finalized 2024) provide.

Harvest now, decrypt later. An adversary recording encrypted traffic today can decrypt it whenever a cryptographically relevant quantum computer exists. For data with a long confidentiality lifetime — health records, state secrets, provenance chains meant to hold for decades — the migration deadline is not the arrival of the machine. It is *now minus the required secrecy duration*. Anything you need confidential in 2050 needs post-quantum protection today.

Key takeaways

- Grover's multi-controlled gates transpile to hundreds of CX; success falls exponentially in gate count. Ancilla-assisted decompositions trade qubits for depth, usually correctly.
- Shor reduces factoring to period-finding; only that step is quantum, and it runs in $O((\log N)^3)$.
- Measuring the second register collapses the first into a periodic comb — a constructive use of measurement.
- The QFT is not a faster FFT: its output is sampled, not read, so it helps only when a single frequency dominates.
- RSA/DH/ECC break exponentially; AES and SHA lose only a quadratic factor. Harvest-now-decrypt- later moves the migration deadline into the present.

Self-check

Q1 *(Objective 7)* — Factor N = 21 with a = 5 by hand, showing each step. Then identify a value of *a* for which the algorithm fails and say what happens next.

Q2 *(Objective 8)* — You maintain hash-chained audit trails intended to remain verifiable for 30 years. Assess your exposure to Shor and Grover, and state your migration priority.

Check your answers

A1 — N = 21, a = 5:

```
 gcd(5, 21) = 1                                    → proceed
5¹ = 5,  5² = 4,  5³ = 20,  5⁴ = 16,  5⁵ = 17,  5⁶ = 1  (mod 21)
→ r = 6
r even ✓        5³ = 20 ≡ -1 (mod 21)  ✗  FAILS the second condition
```

So a = 5 fails at step 4. When a^{r/2} ≡ −1 (mod N), then N divides (a^{r/2}+1) entirely and gcd(a^{r/2}+1, N) = N while gcd(a^{r/2}−1, N) = 1 — both trivial. You learn nothing and must **restart with a different random a.**

Try a = 2:

```
 2¹=2, 2²=4, 2³=8, 2⁴=16, 2⁵=11, 2⁶=1  (mod 21)   → r = 6
r even ✓        2³ = 8 ≢ -1 (mod 21) ✓
gcd(8 - 1, 21) = gcd(7, 21) = 7
gcd(8 + 1, 21) = gcd(9, 21) = 3           21 = 3 × 7  ✓
```

The failure probability is bounded: for N with at least two distinct odd prime factors, a random *a* succeeds with probability ≥ 1/2. So expected restarts are under 2, and the classical outer loop adds only a constant factor — which is why the algorithm's polynomial complexity is unaffected.

A2 — Your exposure splits cleanly, and the split determines priority.

Hash chain integrity — low risk. SHA-256 faces only Grover, reducing preimage search from 2^{256} to ~2^{128}, and collision resistance degrades less than the naive analysis suggests (BHT gives 2^{n/3}, but with prohibitive quantum memory requirements). SHA-256 remains sound. Migrating to SHA-384/512 is prudent, not urgent.

Digital signatures — high risk, and this is the priority. If entries are signed with RSA or ECDSA, Shor breaks those *completely*. The consequence is specific to your threat model: an adversary with a quantum computer could **forge signatures on backdated entries**, retroactively destroying the non-repudiation your chain exists to provide. A 30-year horizon means signatures generated today must resist attack in 2056 — well beyond most credible estimates for cryptographically relevant quantum hardware.

Priority order:

1. **Migrate signatures to ML-DSA or SLH-DSA now.** SLH-DSA (SPHINCS+) is hash-based, so its security reduces to the hash function you already trust — a conservative choice for long-lived archival signatures, at the cost of larger signature sizes.
2. **Re-sign existing archives** with post-quantum signatures before the classical ones become forgeable. Old signatures do not become secure by age.
3. **Adopt hybrid signatures** (classical + PQ) during transition, so a flaw in the newer primitive does not leave you worse off.
4. **Upgrade hashes to SHA-384** opportunistically.

The C2PA connection is direct: provenance manifests are signed assertions with exactly this longevity requirement, and the specification's crypto-agility provisions exist for this migration. A provenance claim that cannot be verified — or worse, can be forged — in 2056 has failed at precisely the moment it was built to matter.